

Assignment 5.1

Jenkins Build Automation with GitHub Webhook / Poll SCM Pipeline style job

Objective

Configure a Jenkins Pipeline job so that a change pushed to the GitHub repository automatically triggers a build, instead of clicking Build Now. Jenkins is notified either instantly by a GitHub webhook or by periodically polling the repository (Poll SCM). When triggered, Jenkins checks out the code, installs and starts Apache, and deploys the website.

Flow

Developer pushes a change → GitHub repository → webhook / Poll SCM → Jenkins (JM) → checkout → install Apache and copy index.html to /var/www/html (deploy).

Configuration summary

Source	GitHub repository
Repository	https://github.com/DV-45/Jenkins-assignment
Branch	main
Jenkins job	Assignment5-WebHook (Pipeline project)
Jenkins URL	http://3.6.126.99:9090 (use your own Jenkins IP and port)
Trigger	GitHub hook trigger for GITScm polling (Option B) or Poll SCM (Option A)
Webhook payload URL	http://<JENKINS-IP>:<PORT>/github-webhook/

Steps

Step 1: Ensure Jenkins prerequisites

Your Jenkins EC2 must have Java, Jenkins and Git installed.

- Java 21 Amazon Corretto
- Jenkins service running (port 8080 or 9090)
- Git installed (Maven is only needed for Maven projects)

Verify on the Jenkins EC2:

```
java -version
sudo systemctl status jenkins
git --version
```

```
[root@ip-172-31-3-227 ~]# java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[root@ip-172-31-3-227 ~]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-10-08 08:23:43 UTC; 36s ago
     Main PID: 28788 (java)
       Tasks: 41 (limit: 1059)
      Memory: 359.4M
         CPU: 20.114s
        CGroup: /system.slice/jenkins.service
                └─28788 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Oct 08 08:23:36 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: [LF]>
Oct 08 08:23:36 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: [LF]> *****
Oct 08 08:23:36 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: [LF]> *****
Oct 08 08:23:36 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: [LF]> *****
Oct 08 08:23:43 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: 2026-10-08 08:23:43.455+0000 [id=34] INFO jenkins.InitReactorRunner$1#onAttained: Completed 3
Oct 08 08:23:43 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: 2026-10-08 08:23:43.489+0000 [id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is full
Oct 08 08:23:43 ip-172-31-3-227.ap-south-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 08 08:23:45 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: 2026-10-08 08:23:45.479+0000 [id=50] INFO h.m.DownloadService$Downloadable#load: Obtained the
Oct 08 08:23:45 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: 2026-10-08 08:23:45.480+0000 [id=50] INFO hudson.util.Retrier#start: Performed the action che
Oct 08 08:23:48 ip-172-31-3-227.ap-south-1.compute.internal jenkins[28788]: 2026-10-08 08:23:48.530+0000 [id=72] WARNING h.n.DiskSpaceMonitorDescriptor#markNodeOffline0

[root@ip-172-31-3-227 ~]# git --version
git version 2.50.1
[root@ip-172-31-3-227 ~]#
```

Step 2: Create the Pipeline job

Set up a new Jenkins Pipeline project for your repo.

- Dashboard → New Item → Pipeline project.
- Name it Assignment5-Pipeline and click OK.
- Source Code Management → Git. Repository URL: <https://github.com/DV-45/Jenkins-assignment>
- Branch Specifier: */main

Jenkins / All / New Item

New Item

Enter an item name

Assignment5-Pipeline

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Build Steps
- Post-build Actions

Repositories

Repository URL:

Credentials: + Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

Step 3: Configure the build trigger

Choose how Jenkins detects changes. Under Build Triggers in the job configuration, use one of the two options.

Option A: Poll SCM

- Tick Poll SCM.
- Schedule: `H/5 * * * *` (checks GitHub every 5 minutes).

Jenkins asks GitHub for changes on this schedule and starts a build only if there is a new commit. No GitHub settings are needed.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts)

☐ Build after other projects are built

☐ Build periodically

☐ GitHub hook trigger for GITScm polling

☒ **Poll SCM**

Schedule:

Would last have run at Thursday, October 8, 2026, 8:37:00 AM Coordinated Universal Time; would next run at Thursday, October 8, 2026, 8:42:00 AM Coordinated Universal Time.

☐ Ignore post-commit hooks

Step 4: Set up the GitHub webhook

GitHub notifies Jenkins instantly when commits are pushed.

- In the GitHub repo go to Settings → Webhooks → Add webhook.
- Payload URL: `http://<JENKINS-IP>:<PORT>/github-webhook/` (for example `http://3.6.126.99:9090/github-webhook/`). Keep the trailing slash.
- Content type: `application/json`
- Which events: Just the push event.
- Keep Active ticked and click Add webhook.

Webhooks / Manage webhook

Settings Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

`http://3.110.37.90:8080/github-webhook/`

Content type *

`application/json`

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

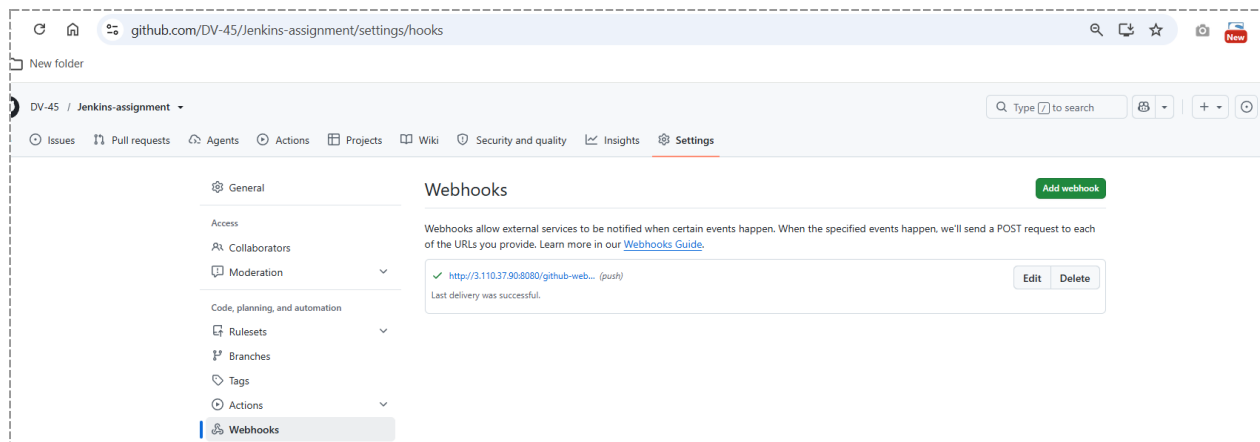
☐ Send me **everything**.

☐ Let me select individual events.

☒ **Active**

We will deliver event details when this hook is triggered.

After adding it, open the webhook and check Recent Deliveries. A green tick means GitHub reached Jenkins.



Notes:

- The Jenkins server's security group must allow inbound TCP on the Jenkins port (8080 or 9090) from anywhere, otherwise GitHub cannot reach it.
- You own this repository, so you can add the webhook. (Adding a webhook needs admin rights on the repo.)

Step 5: Add the build step (install, start, deploy)

The repository holds a static website (demo/index.html), so Jenkins installs and starts Apache and copies the site to the web root. This matches the diagram: install, start, deploy. Do these once on the Jenkins EC2 first:

- Allow the jenkins user to run sudo without a password (needed for yum and systemctl in the build):

```
echo "jenkins ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/jenkins
```

- Open port 80 (HTTP) in the Jenkins EC2 security group so the site can be viewed.

Then in the job: Add pipeline script:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/DV-45/Jenkins-assignment.git'
      }
    }

    stage('Install Apache') {
      steps {
        sh 'sudo yum install httpd -y'
      }
    }
  }
}
```

```

stage('Start Apache') {
    steps {
        sh '''
            sudo systemctl enable httpd
            sudo systemctl start httpd
            '''
    }
}

stage('Deploy') {
    steps {
        sh '''
            sudo cp -r demo/* /var/www/html/
            sudo chown -R apache:apache /var/www/html
            sudo chmod -R 755 /var/www/html
            '''
    }
}

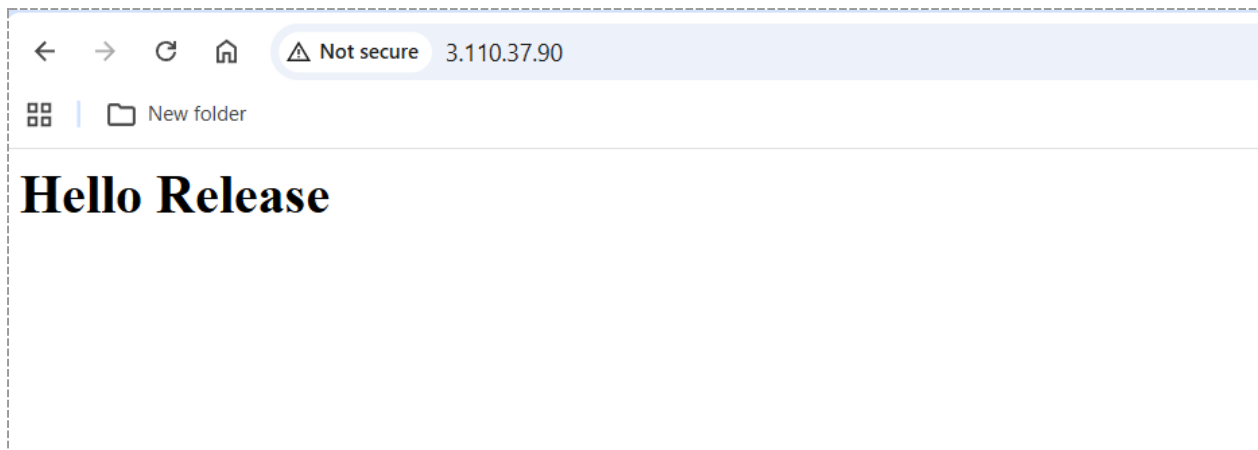
stage('Verify') {
    steps {
        sh 'curl -I http://localhost'
    }
}

}

post {
    success { echo 'Deployment successful' }
    failure { echo 'Deployment failed' }
}
}

```

If index.html is in the repository root instead of the demo folder, change demo/* to * in the copy command. Click Save, then run Build Now once to confirm the job works before testing the automation.



Step 6: Test the automation

Verify that commits trigger Jenkins automatically.

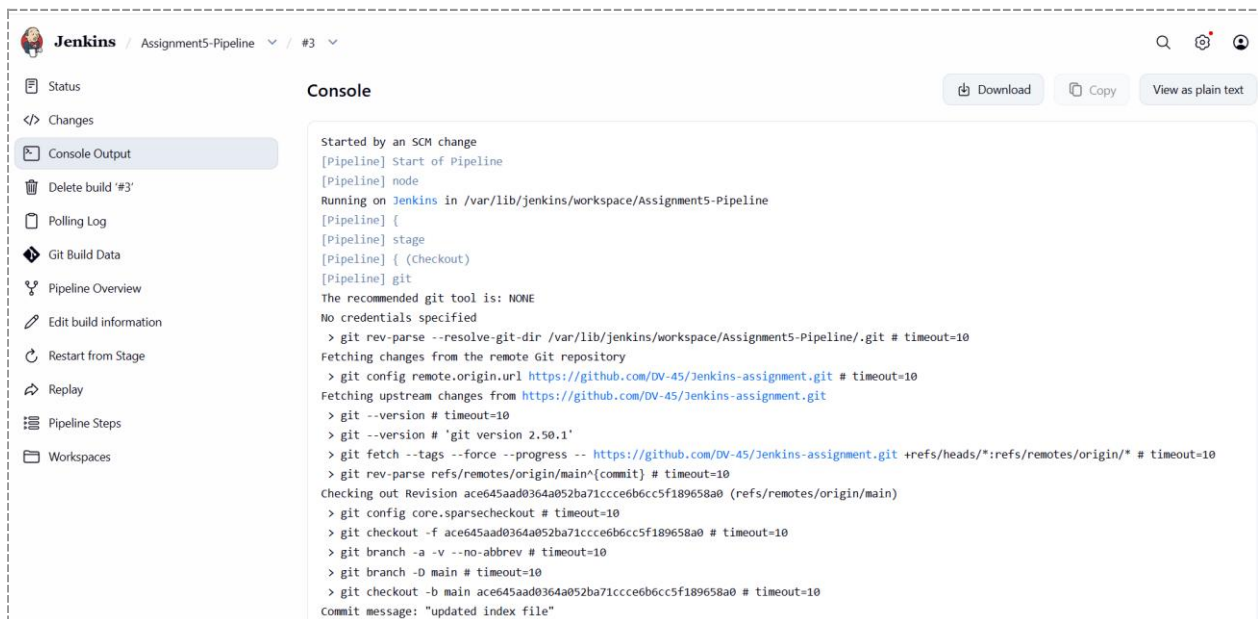
- Push a change to GitHub (for example edit demo/index.html, commit it, and run git push origin main).
- Jenkins job should start automatically (with Poll SCM, within the schedule interval).
- Open the new build → Console Output. It should say "Started by GitHub push" (webhook) or "Started by an SCM change" (poll) and end with Finished: SUCCESS.

```

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\darsh\OneDrive\Desktop\GitRepos\Project\Jenkins-assignment\demo> git add .
PS C:\Users\darsh\OneDrive\Desktop\GitRepos\Project\Jenkins-assignment\demo> git commit -m "updated index page"
[main 4b508c6] updated index page
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\darsh\OneDrive\Desktop\GitRepos\Project\Jenkins-assignment\demo> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 391 bytes | 391.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DV-45/Jenkins-assignment.git
   cb74e3c..4b508c6  main -> main
PS C:\Users\darsh\OneDrive\Desktop\GitRepos\Project\Jenkins-assignment\demo> |

```



Jenkins / Assignment5-Pipeline #3

Console [Download] [Copy] [View as plain text]

```

Started by an SCM change
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Assignment5-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Assignment5-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/DV-45/Jenkins-assignment.git # timeout=10
Fetching upstream changes from https://github.com/DV-45/Jenkins-assignment.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/DV-45/Jenkins-assignment.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision ace645aad0364a052ba71ccce6b6cc5f189658a0 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f ace645aad0364a052ba71ccce6b6cc5f189658a0 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main ace645aad0364a052ba71ccce6b6cc5f189658a0 # timeout=10
Commit message: "updated index file"

```

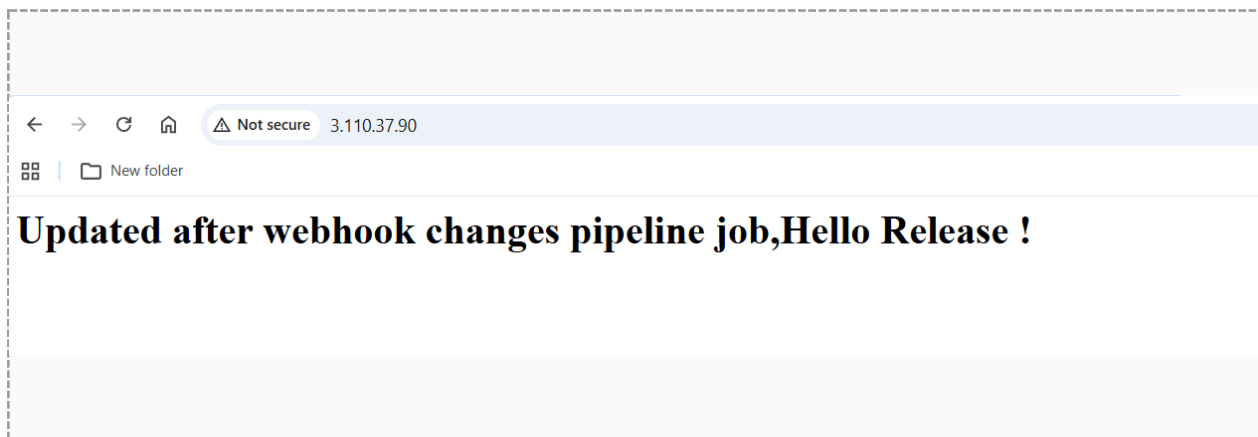
```
Jenkins / Assignment5-Pipeline #3

[Pipeline] sh
+ curl -I http://localhost
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed

0 0 0 0 0 0 0 0
0 100 0 0 0 0 0 0
0 100 0 0 0 0 0 0
0 100 0 0 0 0 0 0

HTTP/1.1 200 OK
Date: Thu, 08 Oct 2026 10:14:16 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Thu, 08 Oct 2026 10:14:15 GMT
ETag: W/"8c-65d517e74b5df"
Accept-Ranges: bytes
Content-Length: 140
Content-Type: text/html; charset=UTF-8

[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Deployment successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Outcome

- A push to GitHub triggers the Jenkins job without any manual action.
- Jenkins checks out the latest code and installs and starts Apache.
- The updated index.html is deployed to /var/www/html and the change is visible in the browser.